I2C Protocol (Inter-Integrated Circuit)

Table of Contents

1. What is I2C
2. Physical layer — wires and electrical characteristics
3. Addressing — how devices are identified
4. Protocol — how data is transferred
5. Clock stretching
6. Pull-up resistors
7. Bus speeds
8. Multi-device bus — how ADS1115 and MLX90640 share
9. I2C on ESP32 — ESP-IDF API
10. ADS1115 over I2C
11. MLX90640 over I2C
12. Debugging I2C issues
13. Common mistakes
14. Glossary

What is I2C

I2C (Inter-Integrated Circuit, pronounced "I-squared-C" or "I-two-C") is a serial communication protocol for connecting multiple devices using only two wires. It was designed by Philips in 1982 for connecting low-speed peripherals to a microcontroller.

Why I2C for SmartDB

SmartDB uses I2C to connect:

- **ADS1115** — 16-bit ADC reading the CT clamp signal
- **MLX90640** — thermal camera

Both devices support I2C natively, and using the same two-wire bus for both keeps wiring simple and conserves ESP32 GPIO pins.

I2C vs SPI vs UART

Protocol	Wires	Speed	Devices	Use case
I2C	2 (SDA + SCL)	Up to 3.4MHz	Multiple on same bus	Sensors, ADCs, small peripherals
SPI	4+ (MOSI, MISO, SCK, CS)	Up to 80MHz	Multiple (one CS per device)	Fast peripherals, displays, flash
UART	2 (TX + RX)	Up to ~5Mbps	Point-to-point only	Debug, GPS, Jetson link

I2C's advantage is **fewer wires and multiple devices on the same bus**. Its disadvantage is **lower speed than SPI** — fine for ADS1115 (860SPS) and MLX90640 (16 frames/sec at 400kHz), not suitable for high-speed data.

Physical layer

The two wires

I2C uses exactly two signal wires:

- **SDA** (Serial Data) — carries data bits
- **SCL** (Serial Clock) — carries the clock signal generated by the master

Both wires are **bidirectional** — any device can pull either line low. This is what makes multi-master and clock stretching possible.

Open-drain / open-collector

I2C lines are **open-drain** — devices can only pull a line LOW (to GND). They cannot drive a line HIGH. The line goes HIGH passively through a **pull-up resistor** connected to VCC.

```
VCC
 |
 R (pull-up resistor)
 |
SDA ——— Device A ——— Device B ——— Device C
         (can pull       (can pull      (can pull
          low)            low)           low)
```

This design is why multiple devices can share the same bus — any device pulling the line low doesn't fight against another device trying to drive it high. It's always the pull-up resistor that drives it high, passively.

A logic HIGH on SDA or SCL means: nobody is pulling it low. A logic LOW means: at least one device is pulling it low.

Addressing

Every I2C device has a **7-bit address** — a unique identifier on the bus. When the master wants to communicate with a specific device, it sends that device's address at the start of every transaction. Only the addressed device responds; all others ignore the transaction.

7-bit address space

7 bits = 128 possible addresses (0x00 to 0x7F). Some addresses are reserved, giving about 112 usable device addresses.

SmartDB device addresses

Device	Address	Set by
ADS1115	0x48	ADDR pin tied to GND
MLX90640	0x33	Fixed in hardware, not configurable

These two addresses don't conflict — they can coexist on the same bus.

ADS1115 address configuration

The ADS1115 has an ADDR pin that lets you choose from 4 possible addresses:

ADDR pin connection	I2C address
GND	0x48
VDD	0x49
SDA	0x4A
SCL	0x4B

SmartDB ties ADDR to GND → address 0x48.

10-bit addressing

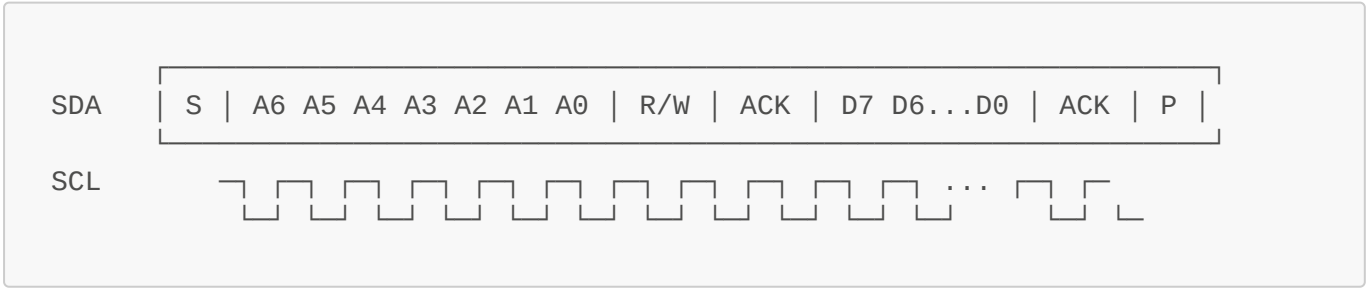
Some devices use 10-bit addresses (1024 possible addresses) for larger systems. Neither ADS1115 nor MLX90640 use 10-bit addressing — you don't need to worry about this.

Protocol

Transaction structure

Every I2C transaction follows this sequence:

```
START → ADDRESS (7 bits) → R/W bit → ACK → DATA → ... → STOP
```



START condition — SDA goes LOW while SCL is HIGH. This signals all devices that a transaction is beginning. Only the master generates START.

Address byte — 7-bit device address followed by 1 R/W bit:

- R/W = 0 → master wants to WRITE to the device

- R/W = 1 → master wants to READ from the device

ACK (Acknowledge) — after each byte, the receiver pulls SDA LOW for one clock cycle to confirm it received the byte. If no ACK is received, the master knows the device is not present or not ready.

NACK (Not Acknowledge) — if the receiver does NOT pull SDA low during the ACK slot, that's a NACK. Means: device not found, address wrong, device busy, or transfer complete (on a read, the master sends NACK after the last byte to signal it has received everything it needs).

STOP condition — SDA goes HIGH while SCL is HIGH. Signals end of transaction. The bus is free for the next transaction.

Repeated START — a new START condition without a preceding STOP. Used in write-then-read transactions (write register address, then read data from it) without releasing the bus between.

Write transaction (master → device)

```
Master: START → [0x48, WRITE] → [register address] → [data byte(s)] → STOP
Device:                  ACK              ACK               ACK
```

Read transaction (device → master)

```
Master: START → [0x48, READ] → NACK/STOP
Device:                  ACK → [data byte(s)]
```

In practice, a read is almost always preceded by a write to set the register address, using a repeated START:

```
Master: START → [0x48, WRITE] → [register address] → RESTART → [0x48, READ]
→ STOP
Device:                   ACK              ACK                          ACK
→ [data]
```

Clock stretching

Clock stretching is when a slave device holds SCL LOW to pause the transaction — telling the master "I'm not ready for the next byte yet, wait."

The master must monitor SCL and wait until the slave releases it (lets it go HIGH) before continuing.

Why it matters for SmartDB

MLX90640 uses clock stretching extensively — it stretches the clock while processing thermal data internally before the data is ready to read. If your I2C master doesn't support clock stretching, reads from MLX90640 will fail silently or return corrupted data.

ESP32's I2C driver supports clock stretching by default — but there is a configurable timeout. If MLX90640 stretches longer than the timeout, the ESP32 will abort the transaction with an error. The default timeout is usually sufficient, but if you see intermittent read failures on MLX90640, increasing the I2C timeout is the first thing to try.

Pull-up resistors

Pull-up resistors are **required** for I2C to function. Without them, SDA and SCL have no way to return to HIGH when no device is pulling them low.

Calculating pull-up value

The correct pull-up value depends on:

- Bus speed (higher speed → lower resistance needed)
- Bus capacitance (longer wires → higher capacitance → lower resistance needed)
- VCC level (3.3V or 5V)

For SmartDB (3.3V, 400kHz, short PCB traces):

```
Recommended: 4.7kΩ
Acceptable range: 2.2kΩ – 10kΩ
```

Too high (>10kΩ): signal rise time too slow, edges are rounded, bit errors Too low (<1kΩ): too much current drawn, devices may not pull line low reliably

Pull-up location

Pull-ups go between VCC and each signal line (one for SDA, one for SCL). They should be placed **once on the bus**, not once per device.

SmartDB pull-up situation

Both the Adafruit MLX90640 breakout board and most ADS1115 module boards include onboard pull-up resistors. Two sets of 10kΩ pull-ups in parallel give an effective resistance of 5kΩ, which is acceptable for 400kHz operation.

Do not add additional external pull-ups — three sets of pull-ups in parallel would drop below 3.3kΩ, which may cause issues at 400kHz.

Bus speeds

I2C defines four standard speed modes:

Mode	Speed	Use case
Standard	100kHz	Old/slow devices
Fast	400kHz	Most modern sensors

Mode	Speed	Use case
Fast-plus	1MHz	High-speed sensors
High-speed	3.4MHz	Rare, specialized

SmartDB bus speed

400kHz (Fast mode) — required by MLX90640 for full 16 fps frame rate.

At 100kHz, MLX90640's internal frame rate would be limited. At 400kHz, you can read a full 32×24 frame in approximately 3ms.

ADS1115 supports up to 400kHz — no conflict.

Bus speed calculation for MLX90640 frame read

MLX90640 frame = 2 × 834 bytes (two subframes) = 1668 bytes total

At 400kHz, each byte takes approximately 22.5µs (9 bits × 2.5µs/bit):

```
1668 bytes × 22.5µs/byte = ~37.5ms per full frame
Actual measured time including overhead: ~40-50ms
Maximum frame rate at 400kHz: ~16-20 fps
```

This confirms 400kHz is appropriate for your use case — at 100ms sampling intervals you have plenty of time to read a full thermal frame.

Multi-device bus

Multiple devices share the same SDA and SCL lines. The master addresses each device individually — a transaction to ADS1115 (0x48) is completely ignored by MLX90640 (0x33), and vice versa.

SmartDB I2C bus topology

```
ESP32 GPIO21 (SDA) ──┬──── ADS1115 SDA
                     └──── MLX90640 SDA (via J3 connector)

ESP32 GPIO22 (SCL) ──┬──── ADS1115 SCL
                     └──── MLX90640 SCL (via J3 connector)

VCC (3.3V) ──── 4.7kΩ ──── SDA
VCC (3.3V) ──── 4.7kΩ ──── SCL
(pull-ups provided by module breakout boards)
```

Bus contention

Bus contention occurs when two masters try to drive the bus simultaneously. SmartDB uses only **one master (ESP32)** — no contention is possible.

However, there can be **transaction interleaving issues** if two FreeRTOS tasks try to use I2C simultaneously without a mutex. This is why we defined an `i2c_mutex` in the FreeRTOS knowledge base — it prevents `sensor_sampling_task` and any other task from using I2C at the same time.

I2C mutex usage pattern

```
// ALWAYS acquire mutex before any I2C transaction
if (xSemaphoreTake(i2c_mutex, pdMS_TO_TICKS(100)) == pdTRUE) {

    // safe to use I2C here
    ads1115_read_conversion(&result);

    xSemaphoreGive(i2c_mutex);
} else {
    ESP_LOGW(TAG, "Could not acquire I2C mutex — skipping read");
}
```

ESP32 I2C — ESP-IDF API

ESP32 has two hardware I2C controllers (I2C_NUM_0 and I2C_NUM_1). SmartDB uses **I2C_NUM_0** with GPIO21 (SDA) and GPIO22 (SCL).

Initialization

```
#include "driver/i2c.h"

#define I2C_PORT        I2C_NUM_0
#define I2C_SDA_PIN     21
#define I2C_SCL_PIN     22
#define I2C_FREQ_HZ     400000    // 400kHz fast mode

esp_err_t i2c_init(void) {
    i2c_config_t conf = {
        .mode             = I2C_MODE_MASTER,
        .sda_io_num       = I2C_SDA_PIN,
        .scl_io_num       = I2C_SCL_PIN,
        .sda_pullup_en    = GPIO_PULLUP_DISABLE,  // using external pull-
ups
        .scl_pullup_en    = GPIO_PULLUP_DISABLE,  // using external pull-
ups
        .master.clk_speed = I2C_FREQ_HZ,
    };
    esp_err_t ret = i2c_param_config(I2C_PORT, &conf);
    if (ret != ESP_OK) return ret;
```

```
    return i2c_driver_install(I2C_PORT, I2C_MODE_MASTER, 0, 0, 0);
}
```

Write to a register

```
esp_err_t i2c_write_register(uint8_t device_addr, uint8_t reg, uint8_t
*data, size_t len) {
    i2c_cmd_handle_t cmd = i2c_cmd_link_create();
    i2c_master_start(cmd);
    i2c_master_write_byte(cmd, (device_addr << 1) | I2C_MASTER_WRITE,
true);
    i2c_master_write_byte(cmd, reg, true);
    i2c_master_write(cmd, data, len, true);
    i2c_master_stop(cmd);
    esp_err_t ret = i2c_master_cmd_begin(I2C_PORT, cmd,
pdMS_TO_TICKS(100));
    i2c_cmd_link_delete(cmd);
    return ret;
}
```

Read from a register (write address, repeated start, read data)

```
esp_err_t i2c_read_register(uint8_t device_addr, uint8_t reg, uint8_t *buf,
size_t len) {
    i2c_cmd_handle_t cmd = i2c_cmd_link_create();

    // Write phase — set register address
    i2c_master_start(cmd);
    i2c_master_write_byte(cmd, (device_addr << 1) | I2C_MASTER_WRITE,
true);
    i2c_master_write_byte(cmd, reg, true);

    // Repeated START — switch to read
    i2c_master_start(cmd);
    i2c_master_write_byte(cmd, (device_addr << 1) | I2C_MASTER_READ, true);

    // Read data — NACK on last byte to signal end of read
    if (len > 1) {
        i2c_master_read(cmd, buf, len - 1, I2C_MASTER_ACK);
    }
    i2c_master_read_byte(cmd, buf + len - 1, I2C_MASTER_NACK);

    i2c_master_stop(cmd);
    esp_err_t ret = i2c_master_cmd_begin(I2C_PORT, cmd,
pdMS_TO_TICKS(100));
    i2c_cmd_link_delete(cmd);
    return ret;
}
```

Timeout configuration (important for MLX90640 clock stretching)

```
// Increase I2C timeout to handle MLX90640 clock stretching
// Default is 10ms — MLX90640 may stretch longer during frame processing
i2c_set_timeout(I2C_PORT, 0xFFFFF);  // maximum timeout
```

ADS1115

What it does for SmartDB

ADS1115 is a 16-bit ADC that reads the voltage across the burden resistor (R1, 33Ω), converting the CT clamp's current output into a digital value that ESP32 can process.

ADS1115 register map

Register	Address	Purpose
Conversion	0x00	Result of last ADC conversion
Config	0x01	PGA range, data rate, mux config
Lo_thresh	0x02	Lower threshold for ALERT/RDY pin
Hi_thresh	0x03	Upper threshold for ALERT/RDY pin

Key config register settings for SmartDB

```
// ADS1115 config register: 16-bit value
// Bits [14:12] MUX:  0b100 = AIN0 vs GND (single-ended)
// Bits [11:9]  PGA:  0b010 = ±2.048V range (matches 1.65V peak from CT)
// Bit  [8]     MODE: 0b0   = continuous conversion
// Bits [7:5]   DR:   0b111 = 860 SPS (maximum sample rate)
// Bits [4:0]   COMP: 0b00011 = traditional comparator, active low, non-
latching

uint16_t config = 0x4283;
//                 ||||
//                 |||└─ 860SPS + comparator config
//                 ||└── continuous mode
//                 |└─── ±2.048V PGA
//                 └──── AIN0 single-ended
```

Reading a conversion

```
#define ADS1115_ADDR        0x48
#define ADS1115_REG_CONV    0x00
#define ADS1115_REG_CONFIG  0x01
```

```
float ads1115_read_voltage(void) {
    uint8_t buf[2];

    // Read 2-byte conversion register
    i2c_read_register(ADS1115_ADDR, ADS1115_REG_CONV, buf, 2);

    // Combine bytes into signed 16-bit integer
    int16_t raw = (int16_t)((buf[0] << 8) | buf[1]);

    // Convert to voltage: ±2.048V range, 16-bit signed = 32768 steps
    float voltage = (float)raw * (2.048f / 32768.0f);

    return voltage;
}
```

Converting voltage to current

```
#define BURDEN_RESISTOR_OHMS  33.0f
#define BIAS_VOLTAGE          1.65f   // VCC/2 from bias divider

float voltage_to_current(float voltage) {
    // Remove DC bias, convert to current
    return (voltage - BIAS_VOLTAGE) / BURDEN_RESISTOR_OHMS;
}
```

ALERT/RDY pin usage

ADS1115's ALERT/RDY pin can signal when a new conversion is ready (in continuous mode, it pulses at the sample rate). This allows interrupt-driven reading instead of polling — more efficient for a 100ms cycle.

Configure Hi_thresh = 0x8000 and Lo_thresh = 0x0000 to put ALERT/RDY in "conversion ready" mode:

```
// Set ALERT/RDY as conversion ready pin
uint8_t hi[2] = {0x80, 0x00};
uint8_t lo[2] = {0x00, 0x00};
i2c_write_register(ADS1115_ADDR, 0x03, hi, 2);  // Hi_thresh
i2c_write_register(ADS1115_ADDR, 0x02, lo, 2);  // Lo_thresh
```

MLX90640

What it does for SmartDB

MLX90640 is a 32×24 pixel infrared thermal camera. It detects heat emitted by objects without touching them — it measures the infrared radiation intensity at each pixel and converts it to a temperature.

For SmartDB, it detects hotspots on electrical components — overheating wiring, loose connections, failing components — that the current sensor alone cannot detect.

MLX90640 I2C address

Fixed at **0x33** — not configurable.

Frame structure

MLX90640 stores data as two subframes (subframe 0 and subframe 1), each containing 834 words (16-bit values). A complete thermal image requires both subframes.

Reading a complete frame:

1. Read status register to check if a new frame is ready
2. Read subframe 0 (834 × 2 = 1668 bytes)
3. Read subframe 1 (834 × 2 = 1668 bytes)
4. Process raw data using MLX90640 calibration data into temperatures

Using the Melexis driver library

Do not implement MLX90640's calibration processing yourself — the math (compensating for pixel-to-pixel variation, ambient temperature, emissivity) is complex and Melexis provides an official C library.

In ESP-IDF, add the MLX90640 library as a component:

```
# Clone into your components directory
cd components/
git clone https://github.com/melexis/mlx90640-library.git mlx90640
```

Then use it:

```
#include "MLX90640_API.h"

#define MLX90640_ADDR  0x33
#define EMISSIVITY     0.95f   // typical for electrical components

static paramsMLX90640 mlx_params;
static uint16_t frame_data[834];
static float temperatures[32 * 24];  // 768 pixels

esp_err_t mlx90640_init(void) {
    // Read calibration data from device EEPROM
    uint16_t eeprom_data[832];
    MLX90640_DumpEE(MLX90640_ADDR, eeprom_data);
    MLX90640_ExtractParameters(eeprom_data, &mlx_params);

    // Set refresh rate — 4Hz (reads two subframes at 8Hz, combined = 4fps)
    // Increase to 16Hz if 400kHz I2C is confirmed working
    MLX90640_SetRefreshRate(MLX90640_ADDR, 0x03);  // 4Hz

    return ESP_OK;
}
```

```
esp_err_t mlx90640_read_frame(float *temp_array) {
    // Read both subframes
    for (int subframe = 0; subframe < 2; subframe++) {
        MLX90640_GetFrameData(MLX90640_ADDR, frame_data);
        float ambient_temp = MLX90640_GetTa(frame_data, &mlx_params);
        MLX90640_CalculateTo(frame_data, &mlx_params, EMISSIVITY,
                             ambient_temp, temp_array);
    }
    return ESP_OK;
}
```

MLX90640 and clock stretching on ESP32

MLX90640 heavily uses clock stretching during frame processing. If you see:

- `ESP_ERR_TIMEOUT` during frame reads
- Partial or corrupted frame data
- Random I2C errors specifically on MLX90640

First thing to check: increase I2C timeout (see section 9).

Debugging I2C issues

I2C scanner — find all devices on the bus

Run this at startup to confirm both ADS1115 (0x48) and MLX90640 (0x33) are detected:

```
void i2c_scan(void) {
    ESP_LOGI("I2C_SCAN", "Scanning I2C bus...");
    for (uint8_t addr = 1; addr < 127; addr++) {
        i2c_cmd_handle_t cmd = i2c_cmd_link_create();
        i2c_master_start(cmd);
        i2c_master_write_byte(cmd, (addr << 1) | I2C_MASTER_WRITE, true);
        i2c_master_stop(cmd);
        esp_err_t ret = i2c_master_cmd_begin(I2C_PORT, cmd,
pdMS_TO_TICKS(50));
        i2c_cmd_link_delete(cmd);
        if (ret == ESP_OK) {
            ESP_LOGI("I2C_SCAN", "Found device at 0x%02X", addr);
        }
    }
}
```

Expected output:

```
Found device at 0x33   ← MLX90640
Found device at 0x48   ← ADS1115
```

If a device is missing: check wiring, check power (VCC/GND), check pull-ups.

Common error codes

Error	Meaning	Fix
ESP_ERR_TIMEOUT	Device not responding or clock stretching too long	Check wiring, increase timeout
ESP_FAIL	NACK received — device didn't acknowledge	Wrong address, device not powered
ESP_ERR_INVALID_ARG	Bad parameter in I2C call	Check function arguments

Logic analyzer

For complex I2C issues, a logic analyzer (even a cheap $10 clone) is invaluable — it shows you exactly what's happening on SDA and SCL at the bit level. Recommended: Saleae Logic 2 (software, free) with any USB logic analyzer that supports I2C protocol decoding.

Common mistakes

1. Forgetting pull-up resistors Without pull-ups, SDA and SCL float. The bus may appear to work intermittently but will fail under load. Always verify pull-ups are present.

2. Adding extra pull-ups on top of module pull-ups Module breakout boards already have pull-ups. Adding more in parallel lowers total resistance below the minimum, causing devices to fail to pull the line low. Check your module before adding external pull-ups.

3. Not using a mutex for I2C in FreeRTOS Two tasks accessing I2C simultaneously causes garbled transactions. Always use `i2c_mutex` before any I2C operation.

4. Using GPIO_PULLUP_ENABLE with external pull-ups ESP32's internal pull-ups are 45kΩ — much too high for I2C at 400kHz. If you have external pull-ups (which you do, from the module boards), set `sda_pullup_en = GPIO_PULLUP_DISABLE` and `scl_pullup_en = GPIO_PULLUP_DISABLE`.

5. Not handling clock stretching timeout for MLX90640 Default I2C timeout is too short for MLX90640. Set `i2c_set_timeout()` to the maximum value during initialization.

6. Reading MLX90640 without both subframes One subframe alone is not a complete image — it's half the pixels. Always read both subframes and process them together.

7. Wrong byte order on ADS1115 conversion result ADS1115 sends MSB first. When combining two bytes into a 16-bit integer: `(buf[0] << 8) | buf[1]` — not `(buf[1] << 8) | buf[0]`.

8. Not checking esp_err_t return values Every ESP-IDF I2C function returns an error code. Ignoring these means silent failures — your code reads zero or garbage without knowing why. Always check `ret != ESP_OK` and log the error.

Glossary

Term	Meaning
ACK	Acknowledge — receiver pulls SDA low to confirm byte receipt
ADS1115	16-bit ADC from Texas Instruments, used for CT clamp signal
Bitmask	A pattern of bits used to select or modify specific bits in a register
Bus	The shared pair of wires (SDA + SCL) connecting all I2C devices
Clock stretching	Slave holds SCL low to pause the transaction
Conversion register	ADS1115 register holding the latest ADC result
EEPROM	Non-volatile memory on MLX90640 storing factory calibration data
Emissivity	A material's efficiency at emitting infrared radiation (0–1)
Fast mode	I2C at 400kHz
Frame	One complete set of 32×24 temperature readings from MLX90640
I2C	Inter-Integrated Circuit — 2-wire serial communication protocol
I2C_NUM_0	ESP32's first hardware I2C controller
MSB	Most Significant Bit/Byte — the highest-value bit/byte
Mutex	Mutual exclusion lock protecting shared resources (see FreeRTOS KB)
NACK	Not Acknowledge — no pull-low during ACK slot
Open-drain	Output that can only pull low, relies on pull-up to go high
PGA	Programmable Gain Amplifier — ADS1115 internal amplifier
Pull-up resistor	Resistor connecting signal line to VCC, pulls line high passively
Refresh rate	How often MLX90640 generates a new thermal frame
Register	Named memory location inside a device, accessed by address
Repeated START	New START condition without preceding STOP
SCL	Serial Clock Line — carries the clock signal
SDA	Serial Data Line — carries data
Slave	Device that responds to master's commands
SPS	Samples Per Second — ADS1115 conversion rate
Standard mode	I2C at 100kHz
Subframe	Half of one MLX90640 frame — two subframes = one complete image
Transaction	One complete I2C communication from START to STOP